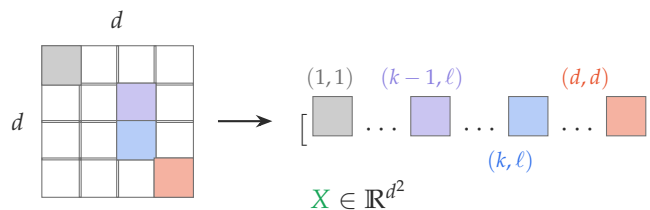


ConvNets

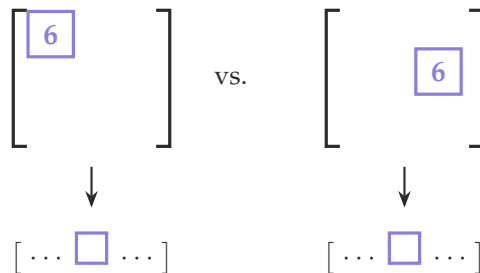
DS 4440 · LECTURE 9

1. So far: simple feed-forward networks

To feed an image to an MLP we flatten it: a $d \times d$ grid becomes a vector in $\mathbb{R}^{d^2}$, and position (k, ℓ) lands at some fixed slot.

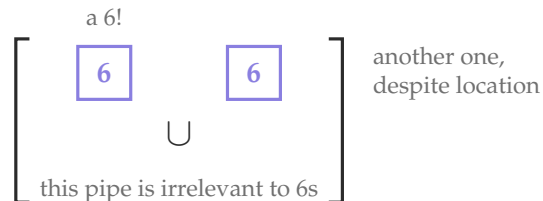


Problem: this discards structure. Nothing in the flattened vector says that two slots were neighbours in the image. Consider the same digit in two places:



The parameters associated with these two 6s are entirely distinct. The model has to learn to spot a 6 once for every location it might appear in.

2. But a 6 is a 6

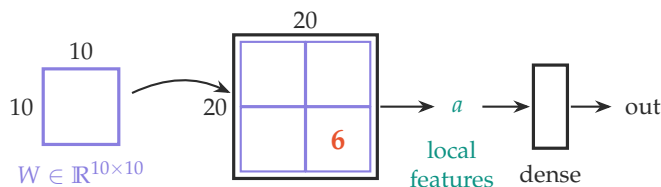


What we want is for the *same* detector to apply everywhere. Two words for the property, and they are not the same:

- **Equivariance**: move the input, and the output moves with it. A convolution is equivariant — shift the 6 and its response shifts too.
- **Invariance**: move the input, and the output does not change. That comes later, from [pooling](#).

3. Enter convolutional neural networks

Basic idea: slide a window over the input to yield **local features**.



Formally

Take a region A of the input starting at (A_x, A_y) . The activation for that region is

$$a_A = \sigma \left(\underbrace{\sum_{i,j} W_{ij} \cdot A_{ij}}_{\text{sum over the window}} \right) = \sigma \left(\sum_{i,j} W_{ij} X_{(A_y+i)(A_x+j)} \right),$$

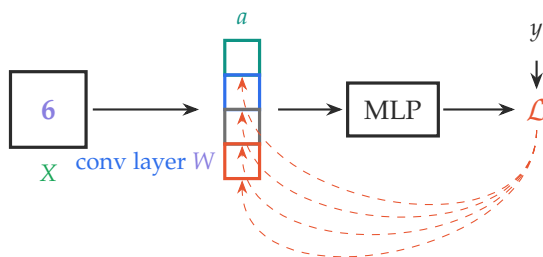
with i indexing rows and j columns in both W and X . For a different region B ,

$$a_B = \sigma \left(\sum_{i,j} W_{ij} \cdot B_{ij} \right).$$

The weights W are **shared across regions**. W is called the **kernel** or **filter**.

Sharing is also what makes this cheap. A 10×10 kernel is 100 weights no matter how large the image is; a dense layer from a 20×20 input to a 20×20 hidden layer is 160,000.

4. What about $\nabla_W \mathcal{L}$ here?



W is used at *every* position, so by the fan-out rule from Lecture 5 it collects one contribution per position, and they add:

$$\nabla_W \mathcal{L} = \sum_{a_k \in a} \left(\nabla_W a_k \cdot \frac{\partial \mathcal{L}}{\partial a_k} \right)$$

This is the shared-weight example from Lecture 5 again, with hundreds of uses instead of two.

A concrete example

(Adapted from “Deep Learning,” Goodfellow et al.) A 3×4 input convolved with a 2×2 kernel:

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \end{bmatrix}_{\text{input}} \otimes \begin{bmatrix} w & x \\ y & z \end{bmatrix}_{\text{kernel}} = \begin{bmatrix} wa + xb & wb + xc & wc + xd \\ + ye + zf & + yf + zg & + yg + zh \\ we + xf & wf + xg & wg + xh \\ + yi + zj & + yj + zk & + yk + zl \end{bmatrix}_{\text{activation}}$$

5. Output size, padding, stride

In the simplest case — as above, no padding and a step of one — the output size is

$$\left(\underbrace{X_h}_{\text{input height}} - \underbrace{W_h}_{\text{kernel height}} + 1, \underbrace{X_w}_{\text{input width}} - \underbrace{W_w}_{\text{kernel width}} + 1 \right).$$

Padding

P	P	P	P	P	P
P	a	b	c	d	P
P	e	f	g	h	P
P	i	j	k	l	P
P	P	P	P	P	P

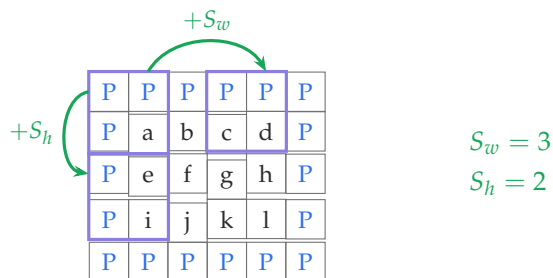
Here we have padded the height and width by 1 on each side, so $P_h = P_w = 2$.

Output shape:

$$(X_h + P_h - W_h + 1, X_w + P_w - W_w + 1)$$

Stride

Stride is how far the kernel moves between positions; we have assumed 1 so far.



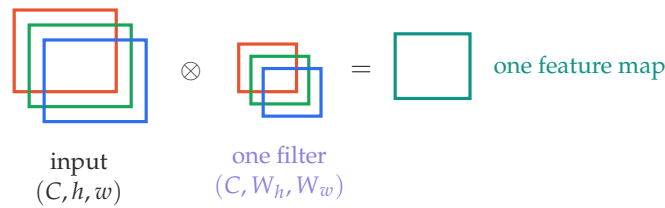
Stride also affects the output size. Counting the positions that fit:

$$\left(\left\lfloor \frac{X_h + P_h - W_h}{S_h} \right\rfloor + 1, \left\lfloor \frac{X_w + P_w - W_w}{S_w} \right\rfloor + 1 \right)$$

The floor matters: with the padded width of 6 above, $W_w = 2$ and $S_w = 3$, we get $\lfloor 4/3 \rfloor + 1 = 2$ positions, not 2.33.

6. Higher dimensions

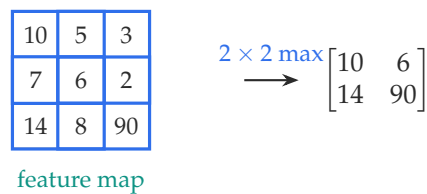
Above we assumed 2-D inputs, but we can extend to arbitrary tensors — useful for RGB images, where the input is (C, h, w) .



Note what is summed and what is not. A single filter spans *all* C channels, and the products are summed over channels as well as over the window — so one filter yields *one* feature map, not one per channel. The depth of the output is the number of filters, not the number of input channels.

7. Pooling

A feature map goes through a **pool** to give a smaller (often scalar) output. **Max pooling** takes the largest value in each window.

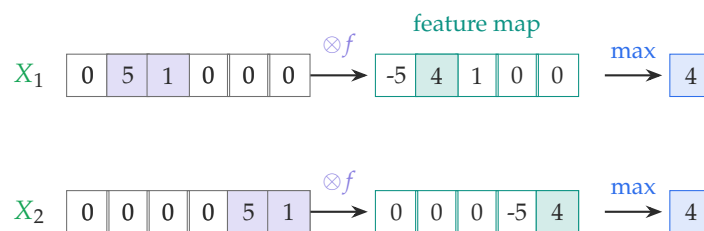


What happens to the gradient through a max?

From Lecture 5: a max is a *router*. The incoming gradient goes entirely to whichever input was the largest, and every other input in the window gets zero. So a pooling layer has no parameters of its own and simply decides which upstream unit hears about the error.

A small example: where invariance comes from

Take a 1-D “image” of six pixels and one filter of width two, $f = [1 \ -1]$, which responds to a bright pixel followed by a darker one. Think of it as the cat-ear detector. Two inputs with the same pattern in different places:



The convolution gives the same response, 4, in both cases — but at position 2 for X_1 and position 5 for X_2 . Flatten those two feature maps and hand them to a dense layer and they look like entirely different inputs, which is the problem we started with. Take the max over the whole map and both collapse to 4: “there is an ear somewhere,” with the location discarded.

The pooling window is the knob. Pool in windows of two instead of globally (letting the last window be short):

$$\text{pool}_2([-5 \ 4 \ 1 \ 0 \ 0]) = [4 \ 1 \ 0], \quad \text{pool}_2([0 \ 0 \ 0 \ -5 \ 4]) = [0 \ 0 \ 4].$$

Still different. Local pooling buys *local* invariance: shift the pattern a little and nothing changes, shift it far and it lands in another pooled cell. Stacking several pooling stages is what widens the region over which position stops mattering.

And the gradient, in the same example

Suppose we are training this as an ear detector and the label for X_1 is “ear.” The loss wants the pooled value higher, so say $\partial \mathcal{L} / \partial p = -0.3$. Walk it back:

- **Through the pool.** The max routes everything to the winner, position 2:

$$\frac{\partial \mathcal{L}}{\partial a} = [0 \quad -0.3 \quad 0 \quad 0 \quad 0].$$

The other four positions used the same filter and get nothing.

- **Through the convolution.** Each position contributes (its incoming gradient) $\times$ (the patch it saw), and only position 2 has a nonzero incoming gradient. That patch is $\begin{bmatrix} 5 & 1 \end{bmatrix}$, so

$$\frac{\partial \mathcal{L}}{\partial f} = -0.3 \begin{bmatrix} 5 & 1 \end{bmatrix} = \begin{bmatrix} -1.5 & -0.3 \end{bmatrix}.$$

- **The step**, with $\alpha = 0.1$: $f : \begin{bmatrix} 1 & -1 \end{bmatrix} \rightarrow \begin{bmatrix} 1.15 & -0.97 \end{bmatrix}$. Run the same patch through again and the response rises from 4 to $1.15 \cdot 5 - 0.97 \cdot 1 = 4.78$.

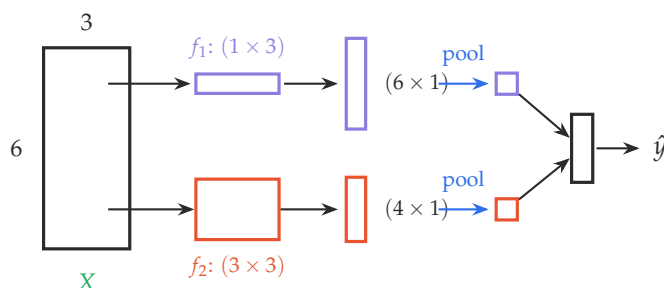
Two things fall out of this. The sum in $\nabla_w \mathcal{L}$ runs over all positions, but max pooling zeroes every term except the winning one — so the filter is only ever trained by the patches that won their windows. And the patch that won X_2 is the same $\begin{bmatrix} 5 & 1 \end{bmatrix}$, at position 5 instead of 2, so the gradient is *identical*. Training on an ear at the left teaches the detector exactly what an ear at the right would. That is weight sharing read backwards.

It is worth asking what the max throws away. The pooled value is 4 whether the image holds one ear or five, and it cannot distinguish an ear on the left from one on the right. Average pooling keeps some of the count and loses the peak. Neither is free.

Intuition. Individual filters may specialize at recognizing something (say, cats). Max pooling is then like asking “does this window contain a cat?” repeatedly, and keeping the strongest answer — which is where *invariance* comes from, as opposed to the equivariance of the convolution itself.

8. Multiple filters

Often we define several independent filters, with the same or varying sizes. Each one produces its own feature map, which is pooled, and the results are concatenated.



Concretely

Take the input below, with no padding, two (1×2) filters, and (1×2) max pooling:

$$X = \begin{bmatrix} 5 & 3 & 1 \\ 6 & 8 & 1 \\ 2 & 3 & 4 \end{bmatrix}, \quad f_1 = \begin{bmatrix} 6 & 8 \end{bmatrix}, \quad f_2 = \begin{bmatrix} -2 & 3 \end{bmatrix}.$$

Each filter sweeps two positions per row, giving a 3×2 feature map:

$$\begin{bmatrix} 6 \cdot 5 + 8 \cdot 3 & 6 \cdot 3 + 8 \cdot 1 \\ 6 \cdot 6 + 8 \cdot 8 & 6 \cdot 8 + 8 \cdot 1 \\ 6 \cdot 2 + 8 \cdot 3 & 6 \cdot 3 + 8 \cdot 4 \end{bmatrix} = \begin{bmatrix} 54 & 26 \\ 100 & 56 \\ 36 & 50 \end{bmatrix} \xrightarrow{\text{pool}} \begin{bmatrix} 54 \\ 100 \\ 50 \end{bmatrix}$$

$$\begin{bmatrix} -2 \cdot 5 + 3 \cdot 3 & -2 \cdot 3 + 3 \cdot 1 \\ -2 \cdot 6 + 3 \cdot 8 & -2 \cdot 8 + 3 \cdot 1 \\ -2 \cdot 2 + 3 \cdot 3 & -2 \cdot 3 + 3 \cdot 4 \end{bmatrix} = \begin{bmatrix} -1 & -3 \\ 12 & -13 \\ 5 & 6 \end{bmatrix} \xrightarrow{\text{pool}} \begin{bmatrix} -1 \\ 12 \\ 6 \end{bmatrix}$$

Concatenating the two pooled maps:

$$[54 \quad 100 \quad 50 \quad -1 \quad 12 \quad 6]$$

which is what we hand to the dense layer. (*More on this in HW4.*)